

Programmation Fonctionnelle

TD n° 10 : Modules et compilation séparée

1 Modules

Le but de cet exercice est d'implémenter un module permettant de représenter et manipuler des ensembles de valeurs de type quelconque. On choisira une implémentation purement fonctionnelle, sans références ni champs mutables.

1. Définir une signature **SET** pour les ensembles, contenant :
 - un type abstrait (pour les ensembles) paramétré par une variable de type (pour les valeurs de l'ensemble)
 - la signature d'une constante **empty** représentant un ensemble vide
 - la signature d'une fonction **add** qui ajoute un élément à un ensemble
 - la signature d'une fonction **element** qui teste si un élément appartient à un ensembleSi vous le souhaitez, vous pouvez ajouter à cette signature la signature d'une fonction **debug** convertissant un ensemble en une liste : ceci vous permettra de savoir ce que contient un ensemble aux différents stades de sa construction.
2. Donner une implémentation **Set** de cette signature, avec représentation des ensembles par des listes. On ne se préoccupera pas de l'ordre des éléments, mais on évitera les doublons.
3. (Optionnel.) Donner une implémentation **Set_abr**, représentant les ensembles par des arbres binaires de recherche. On supposera les éléments d'un ensemble comparables par $<$.
4. Définir :
 - Une signature **SET_USER** pour un utilisateur qui a le droit uniquement d'appeler la fonction **element**,
 - Une signature **SET_ADM** pour un administrateur qui a le droit uniquement de créer de nouveaux ensembles et de leur ajouter des éléments,
 - Deux versions spécialisées du module **Set** appelées **Set_user** et **Set_adm**, de signatures respectives **SET_USER** et **SET_ADM**. Un utilisateur doit pouvoir manipuler une liste créée par un administrateur.

2 Compilation séparée

Le but de cet exercice est de compiler séparément la structure du module **Set**, sa signature, et une unité qui utilise **Set**.

1. Transférez les contenus de la structure et la signature de **Set** dans deux fichiers `myset.ml` et `myset.mli`. Compilez ces deux fichiers avec `ocamlc -c`, en commençant par le `mli` (ceci créera les fichiers `myset.cmi` et `myset.cmo`).
2. Ecrivez et compilez par `ocamlc -c` une unité `prog.ml`, qui utilise l'unité **Myset**. Une fois `prog.ml` intégré à un exécutable, cette unité doit :
 - lire la ligne de commande de l'exécutable,
 - récupérer son 1^{er} argument `v`
 - récupérer les arguments suivants `x_1, ..., x_n`
 - créer à l'aide de **Myset** un ensemble d'entiers contenant `x_1, ..., x_n`.
 - afficher "oui" si `v` est dans l'ensemble, et "non" sinon.

Par exemple :

- la commande `./prog 42 0 3 42 10` doit afficher oui

– la commande `./prog 17 0 3 42 10` doit afficher `non`

Les arguments passés à un exécutable sont dans le tableau de chaînes de caractères `Sys.argv` de la bibliothèque standard : `Sys.argv.(0)` contient toujours le nom de l'exécutable (e.g. `"prog"`), et à partir de la case 1 se trouvent tous les arguments passés, sous forme de chaînes de caractères (e.g. `"42"`).

On peut convertir une chaîne en un nombre à l'aide de la fonction prédéfinie `int_of_string`.

3. Liez par la commande `ocaml` les fichiers `myset.cmo` et `prog.cmo` produits aux étapes précédentes, afin de produire l'exécutable `prog`. Testez l'exécutable.
4. Améliorer le programme précédent en utilisant le module `Arg` de la bibliothèque standard (voir le manuel d'OCaml sur <http://caml.inria.fr>) (valeur `x` passée par option, message si appel incorrect, etc.).